24SJPBERT504 - DATABASE MANAGEMENT SYSTEM

COURSE PROJECT REPORT ON

AGRICONNECT – Marketplace and Crop Advisor

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS & COMPUTER ENGINEERING

Submitted by

JOSEPH BOBY [Reg.No: SJC24ECE046]
KURIAKOSE SEBASTIAN [Reg.No: SJC24ECE049]
NIRANJANA S KUMAR [Reg.No: SJC24ECE055]
SUJITH S [Reg.No: SJC24ECE066]

Guide Name : Mrs. Ancy Mathew
Course Coordinator : Mrs. Pinky M S
Submission Date : 23 / 10 / 2026

ST. JOSEPH'S COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)
CHOONDACHERRY P.O., PALAI – 686579
DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

OCTOBER 2026

ST. JOSEPH'S COLLEGE OF ENGINEERING & TECHNOLOGY
PALAI (AUTONOMOUS)
CHOONDACHERRY P.O., PALAI – 686579
DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING
(An ISO 9001:2015 Certified Institution)

Certificate

This is to certify that the course project report of 24SJPBERT504 - Database Management System, entitled 'AgriConnect – Marketplace and Crop Advisor' is the bonafide work of Mr. Joseph Boby (Reg.No: SJC24ECE046), Mr. Kuriakose Sebastian (Reg.No: SJC24ECE049), Ms. Niranjana S Kumar (Reg.No: SJC24ECE055) and Mr. Sujith S (Reg.No: SJC24ECE066) of the Department of Electronics & Computer Engineering, St. Joseph's College of Engineering & Technology, Palai, in partial fulfilment of the requirements for the Degree of Bachelor of Technology in Electronics & Computer Engineering from APJ Abdul Kalam Technological University, Kerala, carried out under our supervision and guidance.

We hereby certify that the work reported in this project is original and has been carried out by the students mentioned above under our supervision.

Place: Palai
Date: 23 / 10 / 2026

Mrs. Ancy Mathew
Mrs. Pinky M S
Asst. Professors, Dept. of ECE
Course Project Coordinators

Dr. Giby Jose
HoD - ECE
Dept. of Electronics & Computer Engineering

ACKNOWLEDGEMENT

First and foremost, we thank God Almighty for giving us the inner strength, resources, and ability to complete this work successfully, without which all our efforts would have been in vain.

We are grateful to Dr. Giby Jose, Head of the Department of Electronics & Computer Engineering, for his valuable advice and motivation throughout the course of our project.

We sincerely thank our Course Project Coordinators, Mrs. Ancy Mathew and Mrs. Pinky M S (Assistant Professors, Department of Electronics & Computer Engineering), for their constant guidance, valuable advice, and support during the project.

We also record our appreciation for the faculty members of the department and for the technical support extended during the deployment and testing of the application.

Finally, we express our heartfelt gratitude to our parents and friends for their continuous support, and to all those who have directly or indirectly contributed to the successful completion of this project.

ABSTRACT

AgriConnect is a web-based agricultural marketplace backed by a relational PostgreSQL database hosted on Supabase. It connects farmers directly with buyers, reducing dependence on intermediaries and improving price transparency. Farmers can register, publish crop listings with quantity, unit, price and harvest date, monitor stock, and manage incoming orders through a status-based lifecycle. An AFTER INSERT trigger automatically reduces the listed quantity and marks a listing sold out when the stock reaches zero. Buyers can browse, search and filter produce by crop, village and price, and place orders directly with farmers. The database records payments, order-status history, order-linked ratings, government schemes and market price history, and exposes the information through views and stored functions for monthly sales, most-active buyers, farmer ratings, inventory, low-stock alerts, pending payments and farmer revenue. A Crop Advisor function scores crops from a knowledge base against soil pH, moisture, N/P/K, temperature and season, returning match percentages, expected revenue ranges and market outlook. Aadhaar-based login stores only a salted SHA-256 hash, so the raw identity number is never stored or returned. The project demonstrates normalisation, primary and foreign keys, CHECK constraints, triggers, views, stored functions, aggregate queries and role-aware application flow in a working agricultural marketplace.

CONTENTS

SECTION NO.	Title	Page No.
	CERTIFICATE	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	vi
	LIST OF TABLES	vii
	LIST OF ABBREVIATIONS	viii
SECTION 1	**PROJECT OVERVIEW**	**1**
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Existing System	2
1.4	Proposed System	3
1.5	Objectives	4
1.6	Scope	4
1.7	System Requirements	5
1.7.1	Hardware Requirements	5
1.7.2	Software Requirements	5
SECTION 2	**METHODOLOGY**	**7**
2.1	Approach Taken	7
2.2	Tools and Technologies Used	7
2.3	System Architecture	8
2.4	Database Design	9
2.4.1	Entity-Relationship Diagram	10
2.4.2	Table Descriptions	11
2.4.3	Views	13
2.4.4	Stored Functions	13
2.4.5	Triggers and Integrity Constraints	14
2.5	Block Diagram	15
2.6	Component Descriptions	16
i.	Frontend (Next.js)	16
ii.	Backend (FastAPI)	16
iii.	Database (Supabase / PostgreSQL)	17
iv.	Authentication Module	17
v.	Crop Advisor Engine	17
vi.	Orders, Stock and Reporting	17
2.7	Workflow	18
SECTION 3	**RESULT**	**20**
3.1	Implementation Screenshots	20
3.2	Test Results	25
SECTION 4	**REFLECTION AND LEARNING**	**28**
4.1	Challenges Faced	28
4.2	Key Learnings	28
4.3	Suggestions for Improvement	29
4.4	Programme Outcomes, PSOs and SDGs	30
4.5	Conclusion	31
SECTION 5	**REFERENCES**	**32**

LIST OF FIGURES

Fig No.	Title	Page No.
2.1	System Architecture	9
2.2	Entity-Relationship Diagram	10
2.3	Block Diagram	16
2.4	Authentication Flow	18
2.5	Order Lifecycle	19
3.1	Login Page	20
3.2	Home Dashboard (Guest)	21
3.3	Active Listings and Search	22
3.4	Order Tracker	23
3.5	Reports and Analytics	23
3.6	Crop Advisor Recommendations	24
3.7	Profit Calculator	25

LIST OF TABLES

Table No.	Title	Page No.
1.1	Hardware Requirements	5
1.2	Software Requirements	5
2.1	Entity-Relationship Summary	11
2.2	Database Tables	11
2.3	Column-Level Table Structures	11
2.4	Reporting Views	13
2.5	Stored Functions and RPCs	14
2.6	Integrity Constraints and Triggers	15
3.1	Authentication and Access Test Cases	25
3.2	Listings and Marketplace Test Cases	26
3.3	Orders and Stock Test Cases	26
3.4	Crop Advisor Test Cases	26
3.5	Reports and Tools Test Cases	27
4.1	Programme Outcome Mapping	30
4.2	Programme Specific Outcome Mapping	30
4.3	Sustainable Development Goal Mapping	31

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DBMS	Database Management System
ER	Entity-Relationship
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PDF	Portable Document Format
PK	Primary Key
RDBMS	Relational Database Management System
RLS	Row-Level Security
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
3NF	Third Normal Form

SECTION 1

PROJECT OVERVIEW

1.1 Introduction

Agriculture is the backbone of the Indian economy, yet small and marginal farmers often struggle to obtain a fair price for their produce. Long supply chains, limited access to buyers and scattered market information reduce bargaining power and make it difficult to plan cultivation, stock and sales. AgriConnect was developed as a database-centred web application that addresses these problems by providing a direct, transparent marketplace for farmers and buyers, together with advisory and reporting tools.

The application allows a farmer to publish produce listings with quantity, unit, price and harvest date, monitor stock and manage incoming orders. A buyer can browse the live marketplace, search and filter listings by crop, village and price, and place orders. Orders move through a controlled lifecycle (pending, confirmed, delivered, paid or cancelled), and an audit trail records every status change. A database trigger keeps stock consistent by reducing the listed quantity automatically whenever an order is inserted and marking the listing sold out when the quantity reaches zero.

The system is implemented as a three-tier application. A Next.js front end provides the user interface, a FastAPI back end exposes the business operations as REST endpoints, and Supabase hosts the PostgreSQL database. The database contains ten tables, ten reporting views, nine stored functions and one trigger. Row-level scoping is applied in the application queries and views, and Aadhaar-based authentication compares a salted SHA-256 hash rather than storing the raw identity number.

In addition to trading, AgriConnect offers a Crop Advisor that scores crops against soil and climate inputs, a price-history and market-watch module, a profit-and-yield calculator, a scheme explorer, live weather information and downloadable CSV and PDF reports. Together these features demonstrate how a well-designed relational schema, constraints, triggers, views and stored functions can support a practical, data-driven web application.

1.2 Problem Statement

Farmers depend heavily on intermediaries and local traders to sell their produce. This reduces the share of the final price that reaches the farmer, hides market information and makes

transactions difficult to verify. Stock is often tracked manually, so quantities become inconsistent after sales, and there is no reliable record of which order was confirmed, delivered or paid. Buyers face the opposite problem: they have no single place to compare available produce, quality signals and prices.

Agricultural guidance is equally fragmented. Soil-test values, weather, season and crop suitability are usually interpreted separately, while government subsidy and pension information is difficult to locate. Existing general-purpose marketplaces do not combine direct trading with a transactional order lifecycle, auditable status changes, database-generated reports and crop advisory in one platform.

The project therefore addresses the following requirements:

- Provide a single platform where farmers and buyers can trade directly without intermediaries.
- Maintain accurate stock by updating listing quantities automatically when orders are placed.
- Enforce a valid order lifecycle and preserve a complete audit trail of status changes.
- Generate reliable sales, inventory, buyer, rating and revenue reports from the database.
- Offer crop recommendations based on soil, climate and season rather than guesswork.
- Expose price history, government schemes, calculators and weather information in one place.
- Protect identity data by storing only a one-way hash of the Aadhaar number.

1.3 Existing System

In the existing agricultural marketing system, farmers usually sell through local traders, commission agents or weekly markets. Records are maintained in notebooks or spreadsheets, and buyers obtain produce through several layers of distribution. Communication takes place through phone calls and informal messages, so prices and availability are not visible to all participants.

The main limitations of this approach are:

- Dependence on intermediaries reduces the farmer's realisations and weakens price transparency.

- Stock and order records are manual, duplicated and difficult to reconcile after a sale.
- There is no validated order lifecycle or audit trail, so disputes cannot be resolved reliably.
- Market prices, crop suitability and scheme information are scattered across unrelated sources.
- Reporting requires manual compilation and is rarely available at the point of decision-making.
- Identity and contact data are handled informally, without a consistent privacy mechanism.

These limitations point to the need for a transaction-safe, database-backed platform that combines direct trade with structured records, validation, reporting and advisory services.

1.4 Proposed System

AgriConnect is proposed as a web-based marketplace with three user roles: guest, farmer and buyer. Guests may browse listings, reports, the Crop Advisor, the profit calculator, the scheme explorer and weather information, but cannot buy or sell. A farmer can create and update listings, view incoming orders and advance their status. A buyer can search the marketplace, place orders and follow their progress. Access is established through an Aadhaar-based login that resolves the role automatically.

The proposed system combines the following functional modules:

1. User registration and Aadhaar-based authentication with automatic role detection.
2. Farmer and buyer profiles with contact, location and crop information.
3. Marketplace listings with search, crop and village filters, price range and card/table views.
4. Order placement, stock reduction through a database trigger and a validated status lifecycle.
5. Crop Advisor that scores crops against pH, moisture, N, P, K, temperature and season.
6. Price-history tracking, market watch, profit-and-yield calculator and live weather.
7. Government scheme explorer with eligibility information for each farmer.
8. Reporting module with monthly sales, active buyers, farmer ratings, inventory, low-stock and pending-payment views.

9. CSV and PDF export of listings and reports.

Every module reads from or writes to the same PostgreSQL database. Referential integrity, CHECK constraints, triggers, views and stored functions keep the data consistent, while the API layer prevents invalid operations such as over-ordering or an illegal status jump.

1.5 Objectives

The main objective of the project is to design and implement AgriConnect, a relational-database-backed marketplace that connects farmers directly with buyers and demonstrates the practical use of triggers, views, stored functions and aggregate queries in a working application.

The specific objectives are:

1. To design a normalised relational schema for farmers, buyers, listings, orders, payments, ratings, schemes, price history, crops and order-status history.
2. To enforce data integrity using primary keys, foreign keys, CHECK constraints, defaults and unique Aadhaar hashes.
3. To automate stock reduction and sold-out detection with an AFTER INSERT trigger.
4. To validate order-status transitions in the database and record every change in an audit table.
5. To provide reporting views and stored functions for sales, revenue, ratings, inventory and buyer activity.
6. To implement a rule-based Crop Advisor that ranks crops against soil, climate and season.
7. To build a responsive web interface with role-aware access for guests, farmers and buyers.
8. To verify the system with functional, negative and database-level tests and to document the results.

1.6 Scope

The project covers the analysis, database design, implementation, integration, testing and deployment of the AgriConnect web application. It includes the complete SQL schema, seed data, the FastAPI service, the Next.js interface and the analytical features described above. The database is hosted on Supabase and the application is deployed on a NAS and exposed through a secure tunnel.

The scope is deliberately limited to a course project. Payments are recorded for tracking only and are not processed through a payment gateway. There is no real-time chat, notification service, mobile application or machine-learning model; the Crop Advisor is a deterministic scoring function over a curated knowledge base. Market price history is seeded demonstration data rather than a live mandi feed. Identity verification uses a hash comparison inside the application database and is not integrated with the UIDAI Aadhaar API.

1.7 System Requirements

The application is lightweight on the client and runs in a modern browser. The server requires Python, Node.js and network access to Supabase and the external weather and geocoding services. The minimum development and demonstration requirements are listed below.

1.7.1 Hardware Requirements

Table 1.1: Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
Memory	4 GB RAM
Storage	10 GB of available space
Display	Standard monitor (1366 × 768 or higher)
Input devices	Keyboard and mouse
Network	Stable internet connection
Client devices	Desktop, laptop or tablet with a modern browser

Higher specifications improve performance when the development server, database tools and browser are used simultaneously.

1.7.2 Software Requirements

Table 1.2: Software Requirements

Software / Technology	Purpose
Operating system	Windows, Linux or macOS
Front end	Next.js 16, React 19, TypeScript, Tailwind CSS
Back end	FastAPI (Python 3.13), Uvicorn
Database	PostgreSQL hosted on Supabase
Database access	Supabase Python client (PostgREST and RPC)
Authentication	Aadhaar-based login with salted SHA-256 hashes
External services	Open-Meteo weather API, Nominatim geocoding
Code editor	Visual Studio Code
Browser	Google Chrome, Firefox or Microsoft Edge
Version control	Git and GitHub

Software / Technology	Purpose
Reporting	CSV export, fpdf2 PDF generation, pandas

The software stack is intentionally open source and reproducible, so the project can be rebuilt from the schema and source repository without proprietary components.

SECTION 2

METHODOLOGY

2.1 Approach Taken

The project followed an iterative, database-first approach. The first phase identified the actors and use cases, derived the entities and relationships, and normalised the schema. The second phase implemented the tables, constraints, trigger, views and stored functions in PostgreSQL and verified them directly with SQL. The third phase exposed the database through a FastAPI service and built the Next.js interface. The final phase integrated the layers, seeded representative data, tested the roles and negative cases, and deployed the application on the NAS.

The schema was normalised to third normal form. Each table represents one entity, every non-key attribute depends on the whole primary key, and transitive dependencies are removed. Derived values such as an order's value are computed from quantity and price instead of being stored, which avoids update anomalies. Reference data such as crops and market prices is isolated in standalone tables so that it can evolve independently of transactional records.

Database logic was implemented close to the data. The stock update is a trigger rather than application code, so it applies to every insert regardless of which client performs it. Order transitions are validated inside update_order_status(), and every accepted transition is written to order_status_history. Reporting is implemented as views and set-returning functions, which keeps the API thin and ensures that all clients see the same calculations.

The application was then built in layers. The API validates request data, applies role checks and calls Supabase through the server-side key. The front end consumes JSON endpoints and renders role-aware pages. Seed data was used to exercise the marketplace, reports and Crop Advisor, and each page was checked at desktop and mobile widths. Deployment followed a repeatable recipe: copy the source, install dependencies, configure environment variables, start the API and web processes, and expose the site through a secure tunnel.

2.2 Tools and Technologies Used

The implementation uses a modern JavaScript front end, a Python API and a hosted PostgreSQL database. The choice of Supabase allowed the team to work with a real relational database while avoiding local database administration during the course project.

- Next.js 16 with React 19 and TypeScript for the responsive single-page interface.
- Tailwind CSS and custom CSS for the visual system, layout and mobile breakpoints.
- FastAPI and Uvicorn for typed REST endpoints, request validation and background execution.
- Supabase (PostgreSQL) for tables, constraints, views, stored functions, triggers and RLS policies.
- supabase-py with PostgREST for table queries and RPC calls from the API.
- Open-Meteo for live weather and Nominatim for place-name geocoding.
- fpdf2 and Python's csv module for report downloads; pandas for tabular processing.
- Playwright for browser-level verification at desktop and mobile widths.
- Git and GitHub for version control; Visual Studio Code as the editor.
- A NAS with a secure tunnel for hosting the API and web build.

2.3 System Architecture

AgriConnect follows a three-tier architecture. The presentation tier is a Next.js application that runs in the browser and communicates with the API over HTTPS. The application tier is a FastAPI service that validates requests, applies role rules and performs reads and writes through the Supabase client. The data tier is a PostgreSQL database hosted on Supabase, containing the schema, views, functions and trigger described in this report. External weather and geocoding services are called only by the application tier.

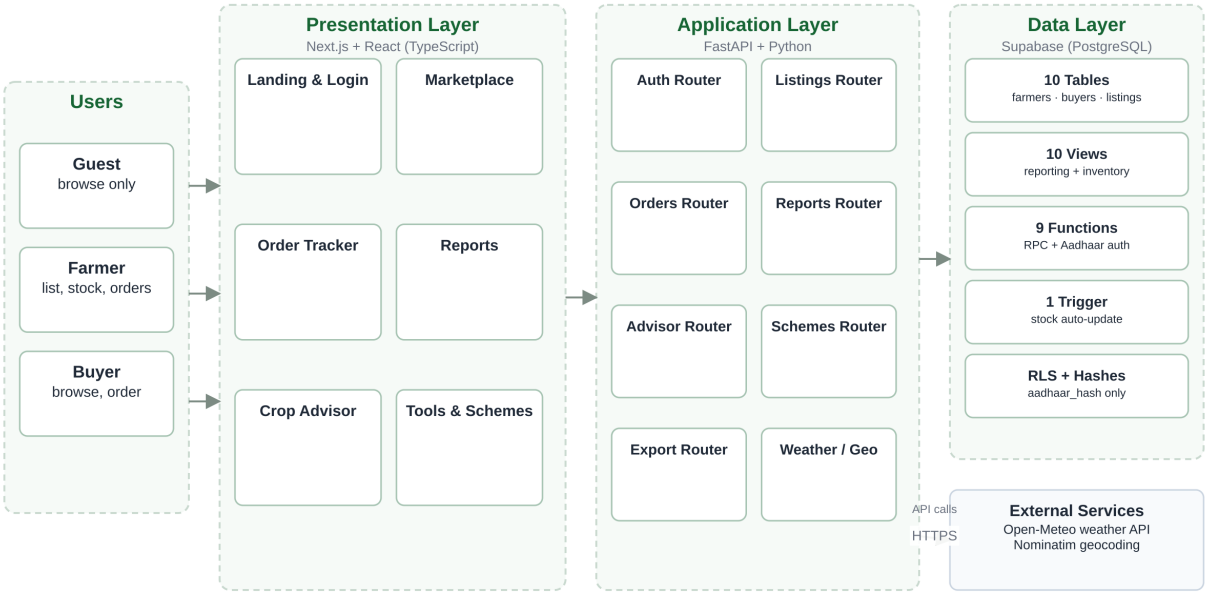


Fig. 2.1: System Architecture

A typical request begins in the browser. The front end sends a JSON request to a FastAPI route, which validates the payload and resolves the current role from the session context. The route then queries a table or view, or calls a stored function through Supabase RPC. The database enforces constraints and triggers before returning the result. The API transforms the result into the response shape expected by the page, and the front end renders it without exposing database credentials.

Separating the tiers provides clear security and maintenance boundaries. The database secret key remains on the server, so it is never shipped to the browser. Reporting logic lives in views and functions rather than being duplicated across pages. The front end can be rebuilt or replaced without changing the schema, and the schema can be tested independently of the interface.

2.4 Database Design

The database is the core of the project. It contains ten tables, ten views, nine stored functions and one trigger. Seven core foreign-key relationships link eight application tables; crops and price_history are standalone reference tables. Primary keys are surrogate identifiers named after the entity (for example, farmer_id and order_id), and every foreign key uses the same naming convention, which makes joins and the ER diagram easy to read.

The schema is normalised to 3NF. Listings store the offered quantity and unit price; order value is derived from quantity_ordered and price_per_unit when needed. Payments and ratings

reference orders rather than duplicating buyer or listing data. The order_status_history table is append-only and records the previous status, the new status, the actor and the timestamp. CHECK constraints restrict statuses, buyer types and rating scores, while UNIQUE constraints protect Aadhaar hashes.

2.4.1 Entity-Relationship Diagram

The entity-relationship diagram below shows the ten entities, their key attributes and the seven foreign-key relationships. Crow's-foot cardinality is one-to-many in every relationship: one farmer has many listings and scheme records, one buyer places many orders, one listing is ordered many times, and one order can have multiple payment, rating and status-history records. The crops and price_history tables are reference data and have no foreign keys.

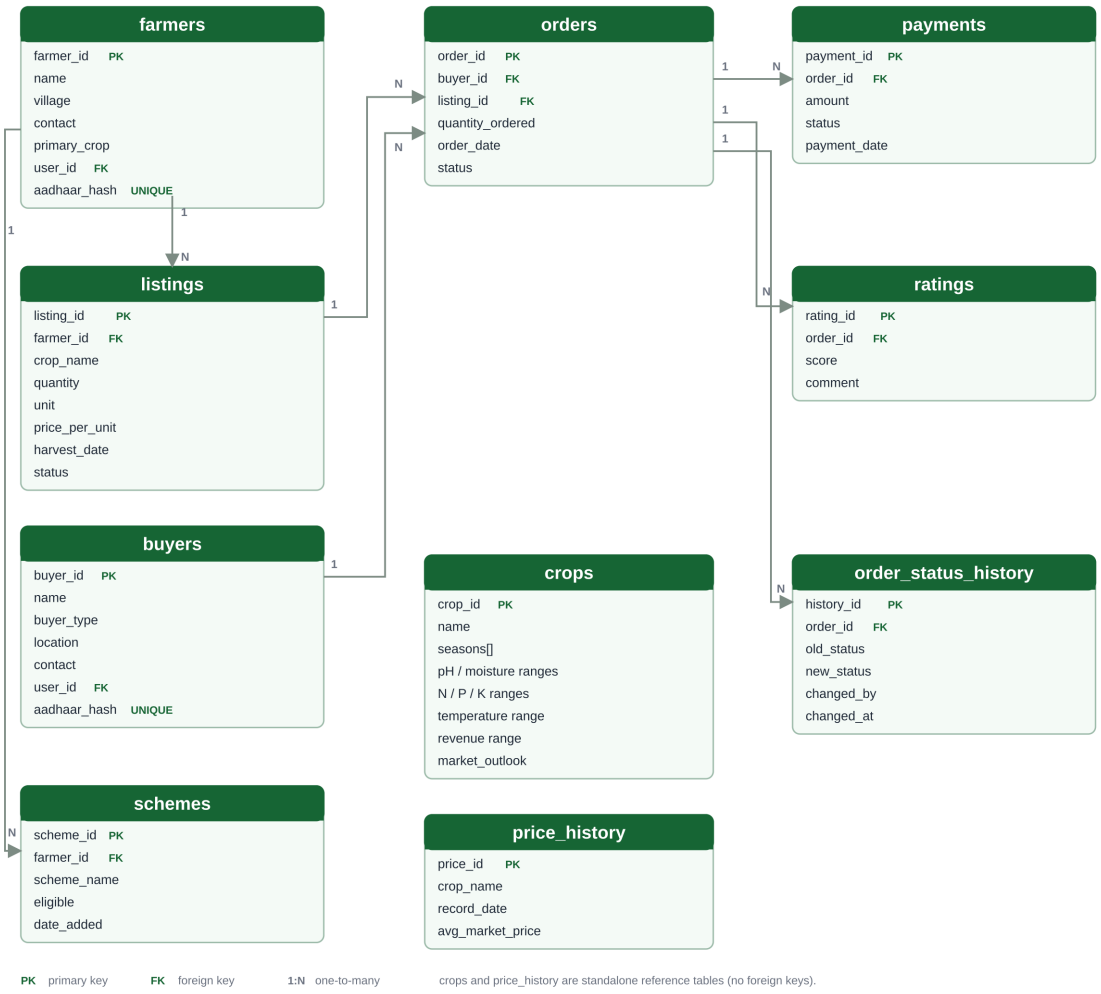


Fig. 2.2: Entity-Relationship Diagram

Table 2.1: Entity-Relationship Summary

Parent	Child	Foreign Key	Cardinality	Description
farmers	listings	listings.farmer_id	1 : N	A farmer publishes many listings.
farmers	schemes	schemes.farmer_id	1 : N	A farmer may have several scheme records.
buyers	orders	orders.buyer_id	1 : N	A buyer places many orders.
listings	orders	orders.listing_id	1 : N	A listing can be ordered many times.
orders	payments	payments.order_id	1 : N	An order can have payment records.
orders	ratings	ratings.order_id	1 : N	A rating is tied to a delivered order.
orders	order_status_history	order_status_history.order_id	1 : N	Audit trail of status transitions.

Standalone reference tables: crops (agronomy knowledge base) and price_history (market prices); neither has a foreign key.

2.4.2 Table Descriptions

Table 2.2 summarises the purpose of each table, and Table 2.3 lists the columns, data types and important constraints. All tables use a SERIAL primary key, and all foreign keys reference the parent primary key. The schema is applied to Supabase as a single SQL script, followed by the seed script that loads demonstration data and the crop knowledge base.

Table 2.2: Database Tables

Table	Purpose
farmers	Registered sellers with name, village, contact, primary crop, optional auth user link and Aadhaar hash.
buyers	Registered purchasers with name, buyer type, location, contact, optional auth user link and Aadhaar hash.
listings	Produce offered for sale: crop, quantity, unit, price per unit, harvest date and availability status.
orders	Purchase transactions linking a buyer and a listing, with quantity, date and lifecycle status.
payments	Payment records for orders, including amount, status and payment date.
ratings	Buyer feedback (1 to 5) linked to an order, with an optional comment.
schemes	Government subsidy or pension schemes associated with a farmer and eligibility status.
price_history	Historical market prices for each crop and record date.
crops	Agronomy knowledge base with season, soil, nutrient, temperature and revenue ranges.
order_status_history	Append-only audit trail of every order status transition.

Table 2.3: Column-Level Table Structures

Table	Column	Data Type	Constraints / Description
farmers	farmer_id	SERIAL	Primary key
farmers	name	VARCHAR(100)	NOT NULL
farmers	village	VARCHAR(100)	Farmer village
farmers	contact	VARCHAR(15)	Contact number

Table	Column	Data Type	Constraints / Description
farmers	primary_crop	VARCHAR(50)	Main crop
farmers	user_id	UUID	FK to auth.users(id)
farmers	aadhaar_hash	TEXT	UNIQUE; salted SHA-256 hash
buyers	buyer_id	SERIAL	Primary key
buyers	name	VARCHAR(100)	NOT NULL
buyers	buyer_type	VARCHAR(20)	NOT NULL; CHECK retailer / wholesaler / consumer
buyers	location	VARCHAR(100)	Buyer location
buyers	contact	VARCHAR(15)	Contact number
buyers	user_id	UUID	FK to auth.users(id)
buyers	aadhaar_hash	TEXT	UNIQUE; salted SHA-256 hash
listings	listing_id	SERIAL	Primary key
listings	farmer_id	INT	FK to farmers
listings	crop_name	VARCHAR(50)	NOT NULL
listings	quantity	DECIMAL(10,2)	NOT NULL; CHECK >= 0
listings	unit	VARCHAR(10)	DEFAULT 'kg'
listings	price_per_unit	DECIMAL(10,2)	NOT NULL
listings	harvest_date	DATE	Date of harvest
listings	status	VARCHAR(20)	DEFAULT 'available'; CHECK available / sold_out
orders	order_id	SERIAL	Primary key
orders	buyer_id	INT	FK to buyers
orders	listing_id	INT	FK to listings
orders	quantity_ordered	DECIMAL(10,2)	NOT NULL
orders	order_date	DATE	DEFAULT CURRENT_DATE
orders	status	VARCHAR(20)	DEFAULT 'pending'; CHECK pending / confirmed / delivered / cancelled / paid
payments	payment_id	SERIAL	Primary key
payments	order_id	INT	FK to orders
payments	amount	DECIMAL(10,2)	NOT NULL
payments	status	VARCHAR(20)	DEFAULT 'pending'; CHECK pending / paid
payments	payment_date	DATE	Date of payment
ratings	rating_id	SERIAL	Primary key
ratings	order_id	INT	FK to orders
ratings	score	INT	CHECK BETWEEN 1 AND 5
ratings	comment	VARCHAR(255)	Optional feedback
schemes	scheme_id	SERIAL	Primary key
schemes	farmer_id	INT	FK to farmers
schemes	scheme_name	VARCHAR(100)	Name of scheme
schemes	eligible	BOOLEAN	DEFAULT FALSE
schemes	date_added	DATE	DEFAULT CURRENT_DATE
price_history	price_id	SERIAL	Primary key
price_history	crop_name	VARCHAR(50)	NOT NULL
price_history	record_date	DATE	NOT NULL
price_history	avg_market_price	DECIMAL(10,2)	NOT NULL
crops	crop_id	SERIAL	Primary key
crops	name	VARCHAR(50)	NOT NULL
crops	seasons	TEXT[]	NOT NULL; Kharif / Rabi / Summer

Table	Column	Data Type	Constraints / Description
crops	ph_min, ph_max	NUMERIC(4,1)	Optimal soil pH range
crops	moisture_min, moisture_max	NUMERIC(5,2)	Optimal moisture range
crops	n_min, n_max	NUMERIC(4,1)	Nitrogen range
crops	p_min, p_max	NUMERIC(4,1)	Phosphorus range
crops	k_min, k_max	NUMERIC(4,1)	Potassium range
crops	temp_min, temp_max	NUMERIC(4,1)	Temperature range
crops	revenue_min, revenue_max	VARCHAR(40)	Expected revenue range
crops	market_outlook	TEXT	Advisory note
order_status_history	history_id	SERIAL	Primary key
order_status_history	order_id	INT	FK to orders; ON DELETE CASCADE
order_status_history	old_status	VARCHAR(20)	Previous status
order_status_history	new_status	VARCHAR(20)	NOT NULL
order_status_history	changed_by	VARCHAR(100)	Actor
order_status_history	changed_at	TIMESTAMPTZ	DEFAULT now()

2.4.3 Views

Views encapsulate joins and aggregations that are reused by the API and the reports page. They keep the SQL readable and ensure that every client applies the same filters and calculations. The ten views are listed below.

Table 2.4: Reporting Views

View	Purpose
active_listings_view	Available listings joined with farmer name and village, ordered for the marketplace.
most_active_buyers_view	Buyers ranked by order count and total purchased volume.
unsold_stock_view	Available stock whose harvest date is older than 15 days.
farmer_avg_rating_view	Average rating and number of ratings for each farmer.
pending_payments_view	Pending payments with buyer and order context.
order_history_view	Full order context with buyer, farmer, crop and computed order value.
order_status_audit_view	Status transition history for every order, newest first.
inventory_view	Current stock position for every listing, including farmer and village.
low_stock_view	Available listings at or below the low-stock threshold of 20 units.
monthly_sales_summary_view	Monthly order count, quantity sold and sales value.

2.4.4 Stored Functions

Stored functions are exposed as Supabase RPCs. They keep multi-step operations atomic and enforce rules that must not depend on the client. The business functions validate transitions and compute reports; the authentication functions hash, store and verify Aadhaar values without ever returning the raw number.

Table 2.5: Stored Functions and RPCs

Function	Arguments	Purpose
recommend_crops	pH, moisture, N, P, K, temperature, season	Scores each crop on six parameters and returns the top five matches with at least four matches.
update_order_status	order_id, new_status, changed_by	Validates the allowed transition, updates the order and writes the audit row.
get_farmer_revenue	farmer_id	Returns the total of paid payments for a farmer.
get_most_active_buyers	none	Ranks buyers by total order volume.
get_farmer_ratings	none	Returns average rating and rating count per farmer.
set_farmer_aadhaar	farmer_id, aadhaar	Validates the 12-digit format and stores a salted hash.
aadhaar_farmer_login	aadhaar	Hashes the input and returns farmer_id and name on a match.
set_buyer_aadhaar	buyer_id, aadhaar	Validates the 12-digit format and stores a salted hash.
aadhaar_buyer_login	aadhaar	Hashes the input and returns buyer_id and name on a match.

The Crop Advisor scoring logic is expressed directly in SQL. The excerpt below shows the six range checks and the threshold that removes weak matches.

```
SELECT c.name,
      ((CASE WHEN p_ph BETWEEN c.ph_min AND c.ph_max THEN 1 ELSE 0 END)
     + (CASE WHEN p_moisture BETWEEN c.moisture_min AND c.moisture_max THEN 1 ELSE 0 END)
     + (CASE WHEN p_n BETWEEN c.n_min AND c.n_max THEN 1 ELSE 0 END)
     + (CASE WHEN p_p BETWEEN c.p_min AND c.p_max THEN 1 ELSE 0 END)
     + (CASE WHEN p_k BETWEEN c.k_min AND c.k_max THEN 1 ELSE 0 END)
     + (CASE WHEN p_temp BETWEEN c.temp_min AND c.temp_max THEN 1 ELSE 0 END))::INT AS match_score
FROM crops c
WHERE p_season = ANY(c.seasons)
ORDER BY match_score DESC
LIMIT 5;
```

Listing 2.1: Crop scoring in recommend_crops()

2.4.5 Triggers and Integrity Constraints

The stock trigger is the most important automated rule in the schema. It fires after every order insert, reduces the listing quantity by the ordered quantity, and updates the status to sold_out when the remaining quantity is zero or negative. Because the rule is enforced inside PostgreSQL, it also protects data written by scripts or future clients.

```
CREATE OR REPLACE FUNCTION update_listing_after_order()
RETURNS TRIGGER AS $$
BEGIN
   UPDATE listings
   SET quantity = quantity - NEW.quantity_ordered
   WHERE listing_id = NEW.listing_id;


   UPDATE listings
   SET status = 'sold_out'
   WHERE listing_id = NEW.listing_id AND quantity <= 0;
   RETURN NEW;
END; $$ LANGUAGE plpgsql;


CREATE TRIGGER after_order_insert
```

```
AFTER INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION update_listing_after_order();
```

Listing 2.2: Automatic stock update trigger

Table 2.6: Integrity Constraints and Triggers

Mechanism	Implementation	Effect
Primary keys	SERIAL primary key on every table	Unique row identity and stable joins.
Foreign keys	Seven core relationships; optional auth user links are separate	Prevents orphan orders, listings and history rows.
CHECK constraints	Status sets, buyer_type, quantity >= 0, score 1-5	Rejects invalid values at write time.
UNIQUE constraint	farmers.aadhaar_hash and buyers.aadhaar_hash	Prevents duplicate identity hashes.
DEFAULT values	unit kg, status available/pending, CURRENT_DATE, now()	Consistent initial values.
Stock trigger	after_order_insert -> update_listing_after_order()	Atomic stock reduction and sold-out flag.
Lifecycle guard	update_order_status() transition rules	Blocks illegal jumps such as pending to delivered.
Audit trail	order_status_history with ON DELETE CASCADE	Every accepted transition is recorded.
Row-level security	Enabled on the core tables	Policies protect access if the anon key is used.

2.5 Block Diagram

The block diagram presents the functional view of the system. Users interact with the authentication, marketplace, order and stock, Crop Advisor, reports, and schemes-and-tools blocks. All blocks read from or write to the AgriConnect database, which acts as the single source of truth. The external weather and geocoding services are used only by the advisory and calculator features.

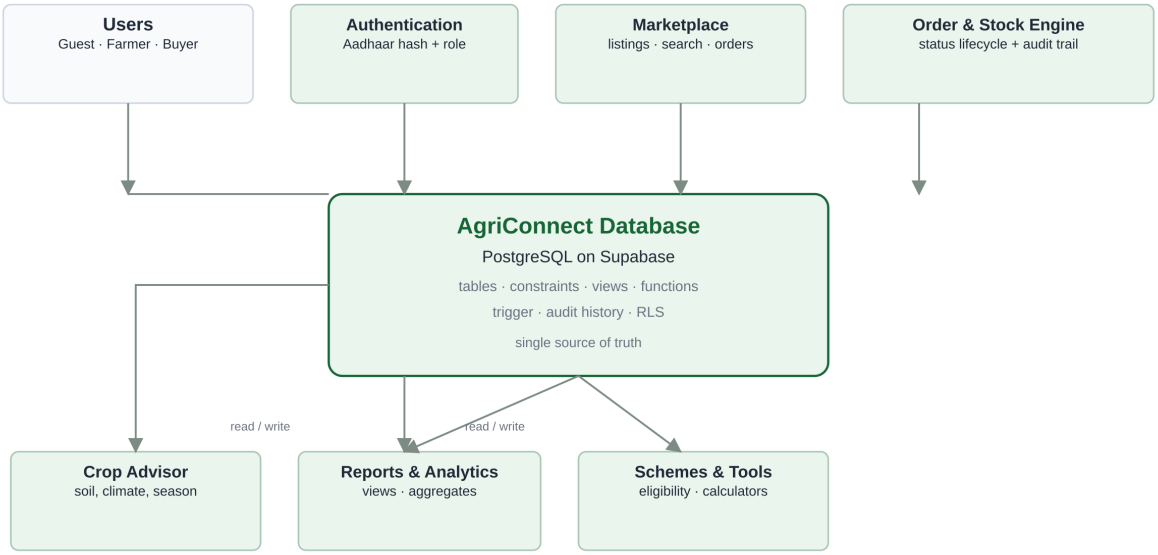


Fig. 2.3: Block Diagram

The authentication block hashes the Aadhaar input and resolves the role. The marketplace block creates listings and performs search and filtering. The order and stock block inserts orders and invokes the stock trigger, then applies validated status transitions. The Crop Advisor block calls recommend_crops() with soil and climate values. The reports block queries the reporting views and functions, and the schemes-and-tools block reads scheme and price data and performs calculator operations.

2.6 Component Descriptions

i. Frontend (Next.js)

The front end is a Next.js application written in TypeScript. It provides the landing and login screen, the marketplace, the order and listing tracker, the reports dashboard, the Crop Advisor, the profit calculator and the scheme explorer. A shared dock allows navigation between these pages, while reusable card, table and form components keep the interface consistent. The layout adapts to desktop and mobile widths, and the application shows role-specific actions for guests, farmers and buyers.

ii. Backend (FastAPI)

The back end is a FastAPI service. Each domain has a router: authentication, farmers, buyers, listings, orders, reports, Crop Advisor, schemes, weather, geocoding, export and marketplace statistics. Routers validate request payloads with Pydantic models, apply role and ownership

checks, call Supabase, and return JSON responses. A small TTL cache reduces repeated database round-trips for statistics and listings.

iii. Database (Supabase / PostgreSQL)

Supabase hosts a PostgreSQL database that stores all transactional and reference data. The schema defines ten tables, constraints, one trigger, ten views and nine functions. The API uses the server-side secret key, so database credentials never reach the browser. Row-level security is enabled on the core tables, and the Aadhaar hash columns are unique and never exposed in query results.

iv. Authentication Module

The authentication module accepts a 12-digit Aadhaar number, validates its format, appends the application salt and computes a SHA-256 hash. It then calls aadhaar_farmer_login and, if no farmer matches, aadhaar_buyer_login. A successful call returns only the entity identifier and name, which the front end stores as the active session. The raw number and the stored hash are never returned or displayed. On registration, set_farmer_aadhaar or set_buyer_aadhaar performs the same validation and stores the hash.

v. Crop Advisor Engine

The Crop Advisor is a rule-based engine, not a machine-learning model. It reads the crops knowledge base and scores each crop against six parameters: pH, moisture, nitrogen, phosphorus, potassium and temperature. The season is applied as a filter. A crop must match at least four of the six parameters to be returned, and results are ranked by match score and limited to five. Each result includes the match percentage, an expected revenue range and a market outlook. Live weather from Open-Meteo and place-name geocoding from Nominatim help the user fill in the climate inputs.

vi. Orders, Stock and Reporting

Orders link a buyer, a listing and a quantity. On insertion, the trigger reduces stock and, if necessary, marks the listing sold out. The API checks that the requested quantity is positive and does not exceed the available stock. Status changes are applied through update_order_status(), which permits only pending to confirmed or cancelled, confirmed to delivered or cancelled, and delivered to paid. Every accepted change is written to the audit table. The reporting module reads the views and functions to produce monthly sales, active

buyers, farmer ratings, inventory, low-stock alerts, pending payments and revenue figures, with CSV and PDF export.

2.7 Workflow

The end-to-end workflow begins with registration and login. A farmer publishes a listing, which immediately appears in the marketplace. A buyer searches or filters the active listings, selects a quantity and places an order. The order insert triggers the stock update, and the farmer advances the order through confirmation and delivery. Once payment is recorded, the order is marked paid. Ratings can then be associated with the order, and all activity feeds the reporting views.

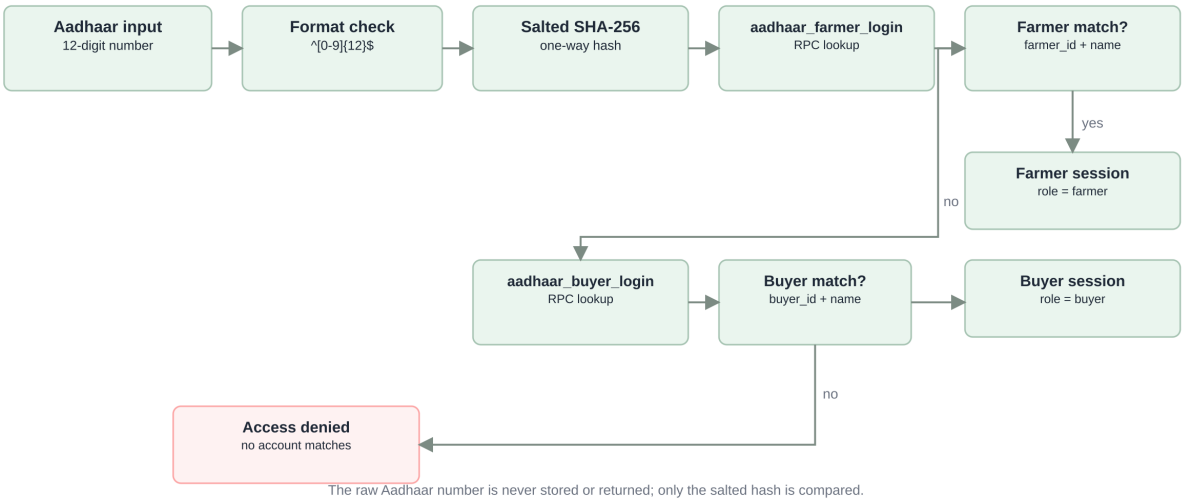


Fig. 2.4: Authentication Flow

Authentication is deliberately simple for the user but strict in the database. The input is validated before any lookup, and the comparison is performed against a hash. The role is resolved without asking the user to choose it, which prevents a user from claiming the wrong role. A failed lookup returns a single generic message, so the response does not reveal whether a hash exists.

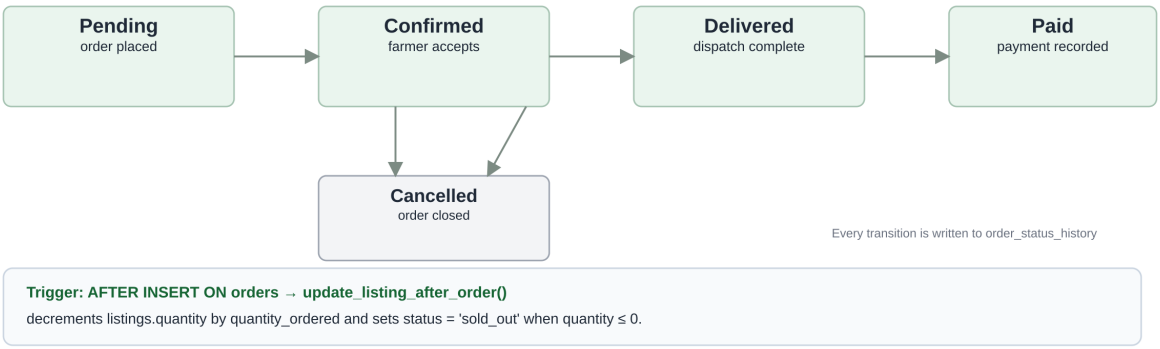


Fig. 2.5: Order Lifecycle

The order lifecycle is a directed graph with a guarded transition function. Only the transitions shown in the figure are accepted; an attempt to move directly from pending to delivered raises an error. The cancelled state closes the order without restoring stock automatically, which is a deliberate limitation of the current implementation and a candidate for future work. The trigger records stock movement at the moment of order insertion, and the audit table records who changed the status and when.

SECTION 3
RESULT

3.1 Implementation Screenshots

The application was implemented and deployed with a seeded dataset of ten farmers, nine buyers, twenty-three listings and twenty orders. The screenshots below show the principal pages at a desktop resolution of 1440 pixels. The interface uses a restrained green palette, rounded cards and a persistent bottom dock so that the marketplace and analytical tools remain one click away.

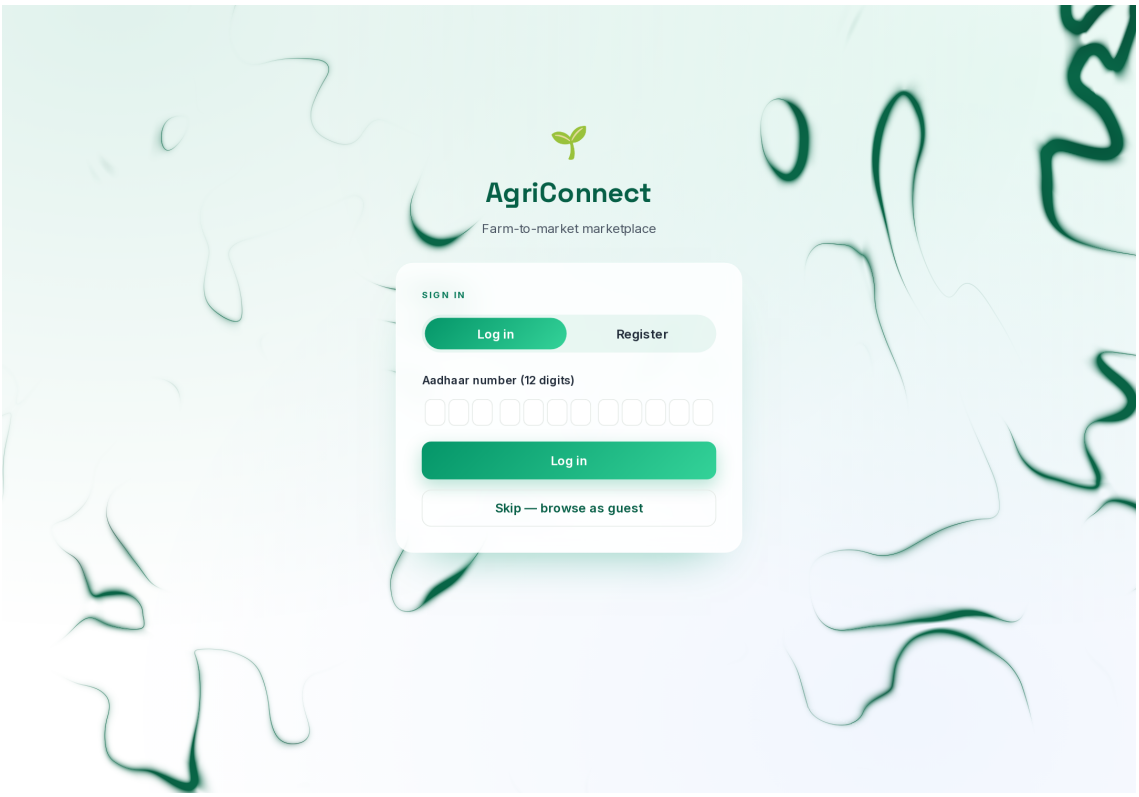


Fig. 3.1: Login Page

The login page offers a twelve-digit Aadhaar input, an explicit registration tab and a guest-browsing option. Guests can explore public listings, reports and tools, but buying and selling actions remain disabled until a farmer or buyer signs in. The guest option was added so that evaluators and first-time visitors can inspect the application without creating accounts.

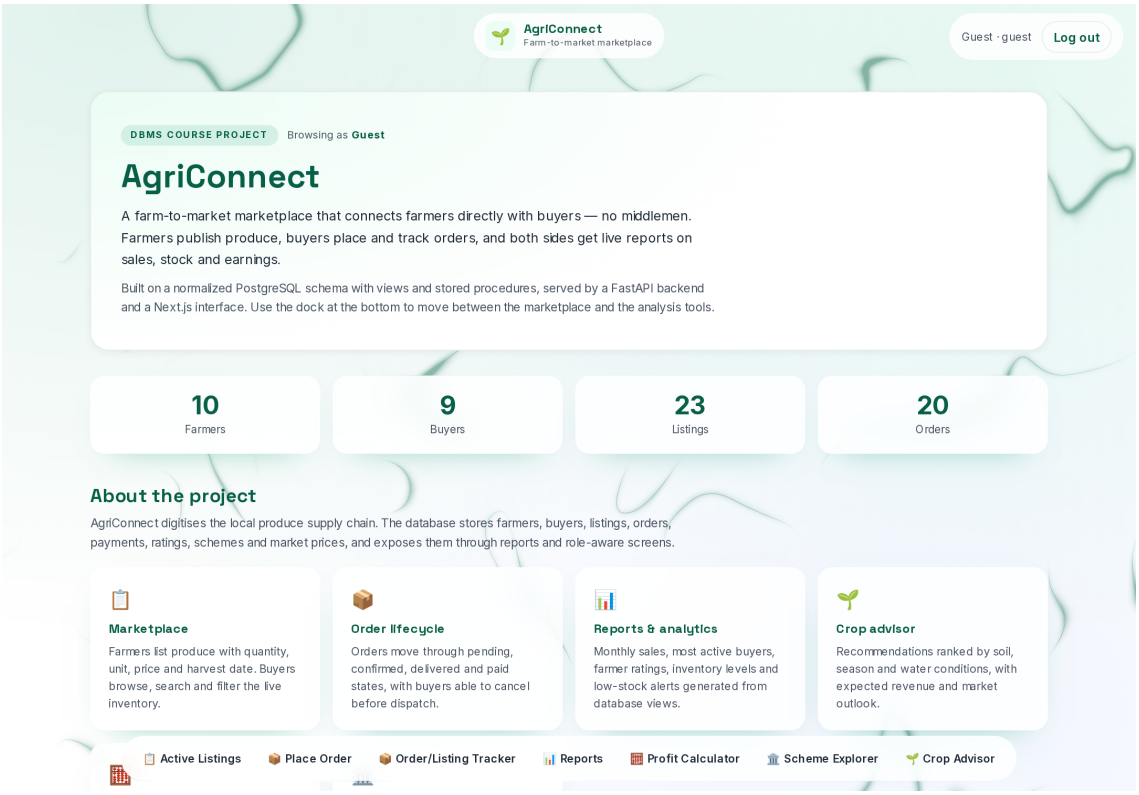


Fig. 3.2: Home Dashboard (Guest)

The home page summarises the project and shows live counts of farmers, buyers, listings and orders. The cards explain the marketplace, order lifecycle, reports and Crop Advisor. The dock at the bottom provides access to every module and highlights the current page.

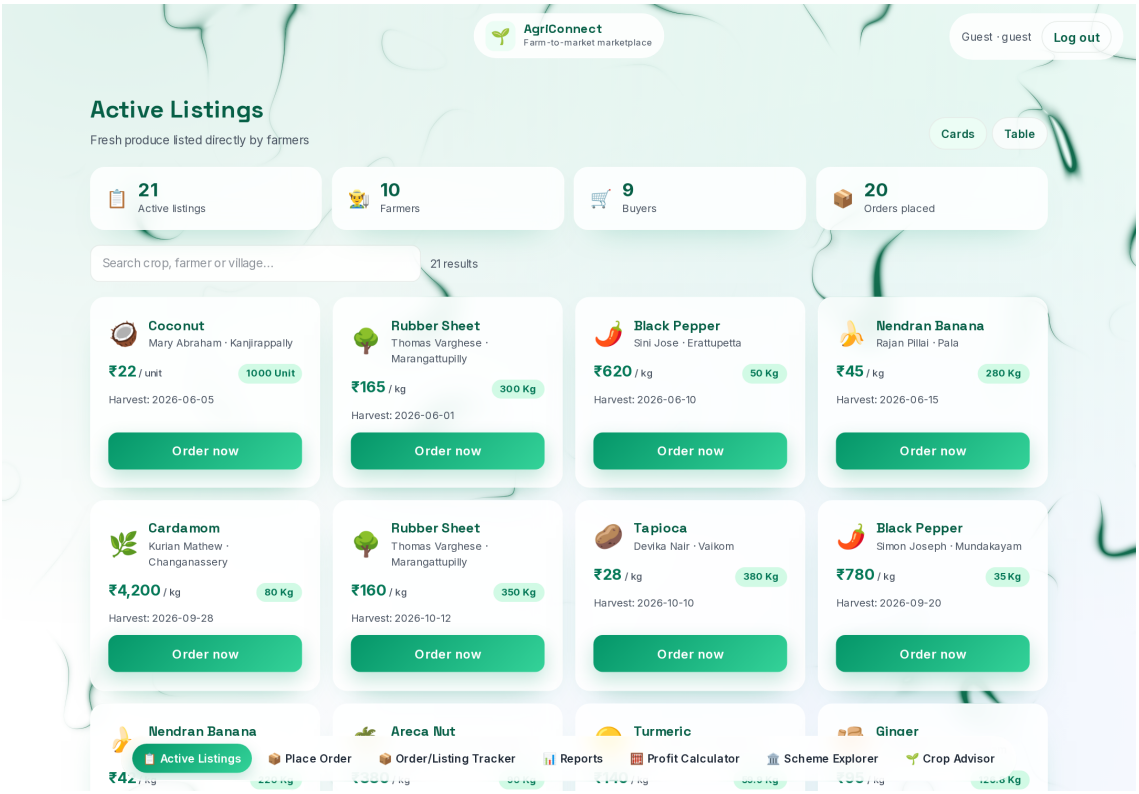


Fig. 3.3: Active Listings and Search

The marketplace displays live listings with crop, farmer, village, price, available quantity and harvest date. Users can switch between card and table views, search by crop, farmer or village, filter by crop and village, restrict the price range and export the current result set to CSV. The page shown here contains twenty-one active listings drawn from the seeded database.

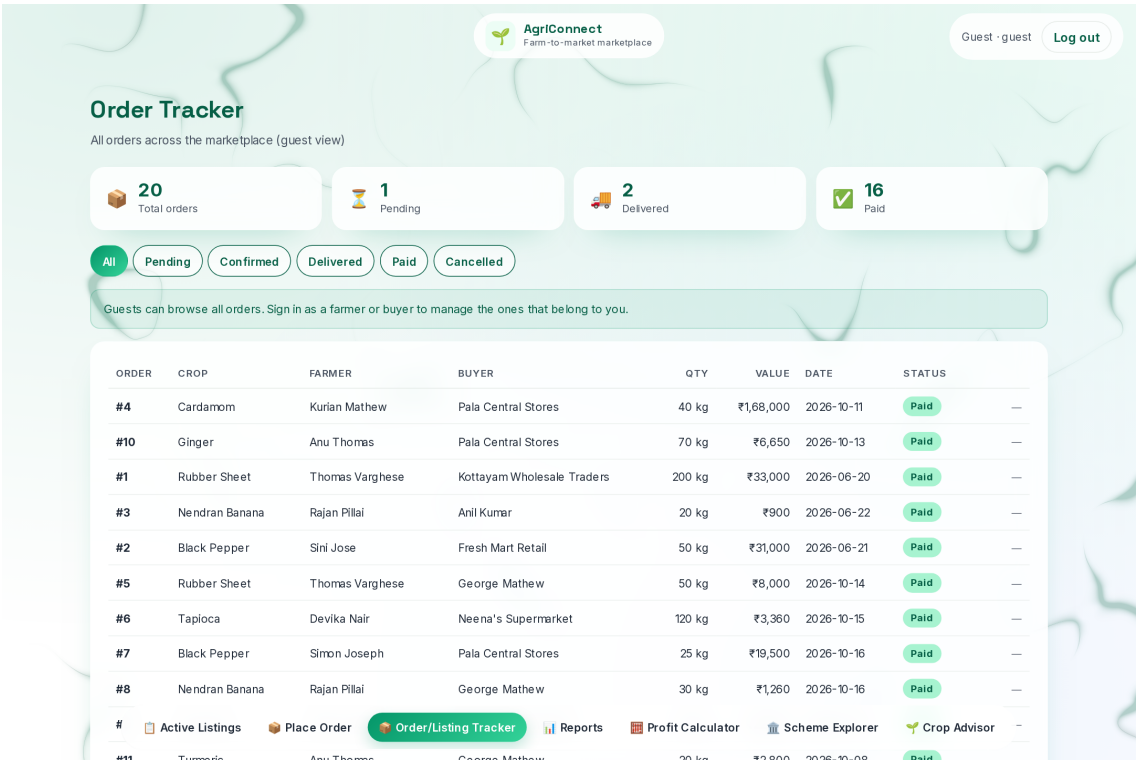


Fig. 3.4: Order Tracker

The order tracker lists orders with crop, farmer, buyer, quantity, value, date and status. Status chips filter the list, and role-aware actions appear only for the farmer or buyer who owns the order. The guest view shown here is read-only and includes a note explaining the restriction.

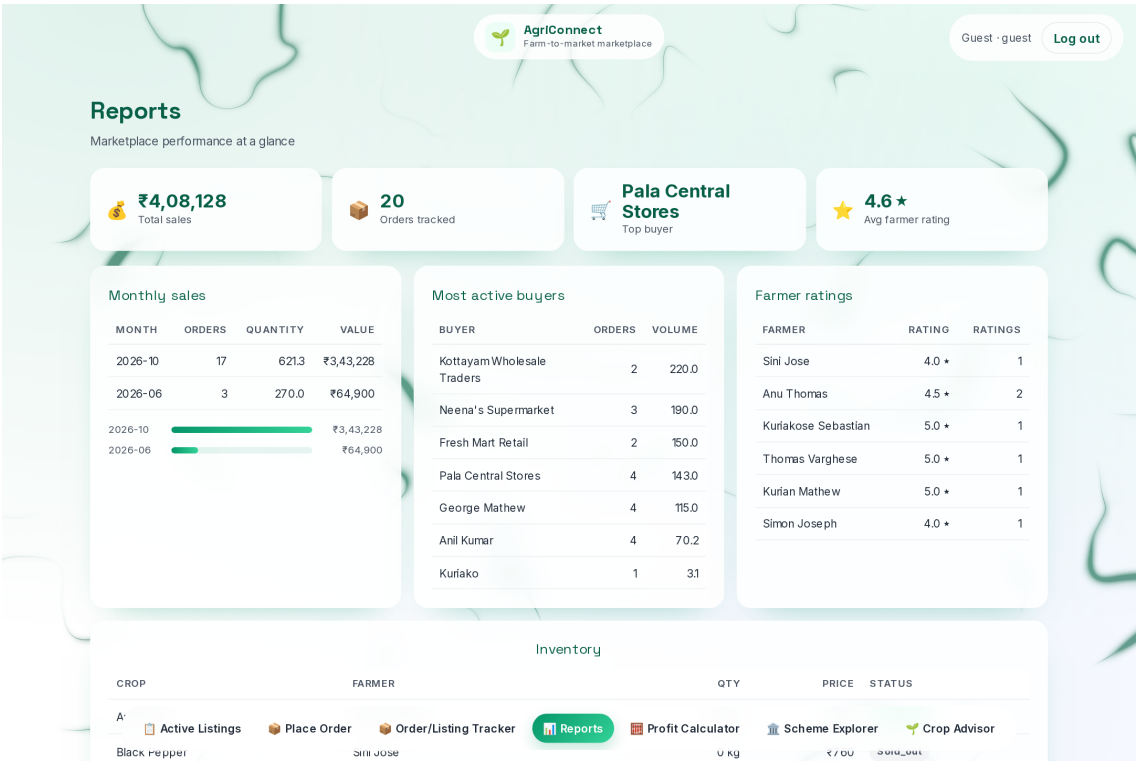


Fig. 3.5: Reports and Analytics

The reports page consolidates monthly sales, most-active buyers, farmer ratings and current inventory, with a low-stock panel below. The figures are produced by the reporting views and functions rather than by client-side aggregation, so the same numbers are available to every consumer of the database.

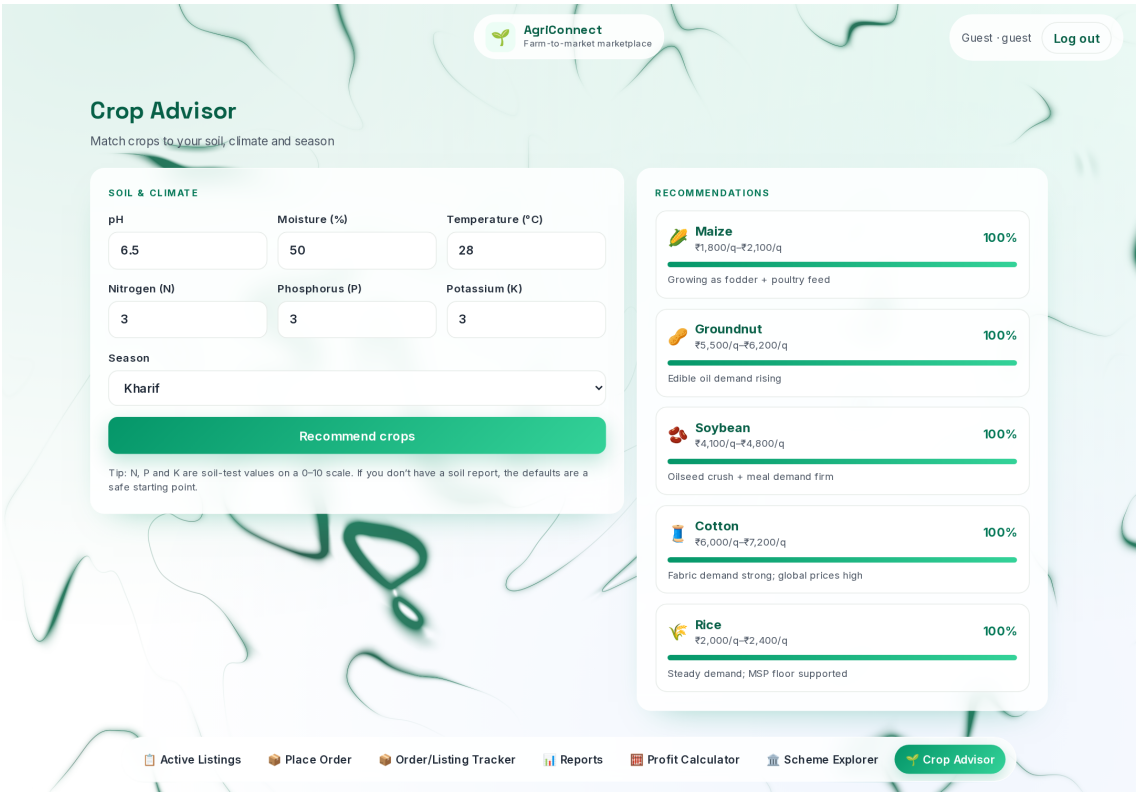


Fig. 3.6: Crop Advisor Recommendations

The Crop Advisor accepts pH, moisture, temperature, nitrogen, phosphorus, potassium and season. The example uses pH 6.5, moisture 50, temperature 28 and N/P/K values of 3 for the Kharif season. The function returns ranked crops with a match percentage, an expected revenue range and a market outlook. Live weather and geocoding can populate the climate inputs.

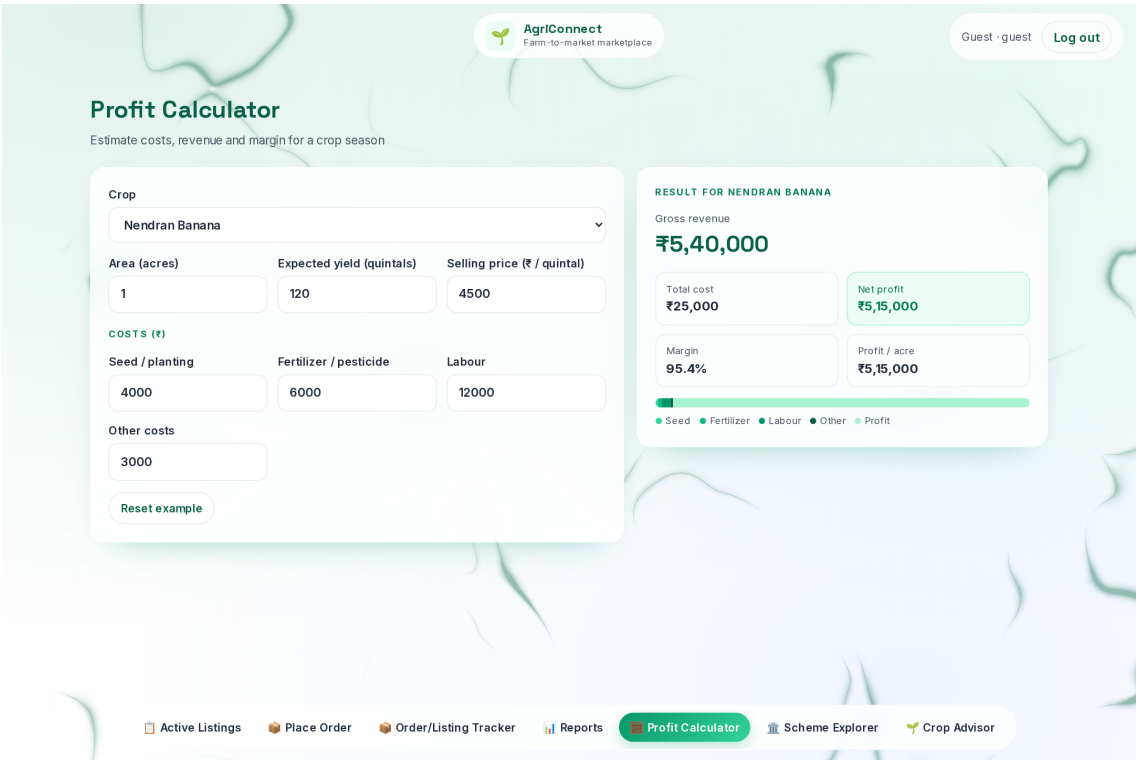


Fig. 3.7: Profit Calculator

The profit calculator combines area, expected yield, selling price and cost components to produce gross revenue, net profit, margin and profit per acre. It is a planning aid, not a database transaction, and it uses the same crop list as the marketplace and the Crop Advisor.

3.2 Test Results

Testing was carried out at three levels. Database logic was verified with SQL checks for the stock trigger, the order transition function, the reporting views and the authentication functions. Page-level functional testing covered guest, farmer and buyer workflows, including negative cases such as an invalid Aadhaar format, over-ordering and illegal status jumps. Finally, the interface was checked with Playwright at desktop and mobile widths (320, 360 and 390 pixels) for layout overflow, console errors and session behaviour.

The tables below record the principal test cases and their outcomes. All 43 cases passed on the deployed build.

Table 3.1: Authentication and Access Test Cases

ID	Test Case	Expected Result	Status
A1	Farmer login with a registered Aadhaar	Logs in with role farmer and shows the name	Pass
A2	Buyer login with a registered Aadhaar	Logs in with role buyer	Pass
A3	Role auto-detection	Correct role resolved without a role picker	Pass

ID	Test Case	Expected Result	Status
A4	Aadhaar with fewer than 12 digits or non-numeric characters	Message: Enter a valid 12-digit Aadhaar number.	Pass
A5	Aadhaar not present in the database	Message: No account matches that Aadhaar. Please register first.	Pass
A6	Guest access	Public pages visible; buy and sell actions blocked	Pass
A7	Raw Aadhaar exposure	Raw number never shown in the UI or logs	Pass
A8	Session persistence after reload	Session restored and re-validated against the database	Pass

Table 3.2: Listings and Marketplace Test Cases

ID	Test Case	Expected Result	Status
B1	Farmer adds a listing	Listing appears in Active Listings	Pass
B2	Guest or buyer tries to add a listing	Action blocked or disabled	Pass
B3	Search for Rice	Only matching crop, farmer or village rows	Pass
B4	Filter by crop Tomato	Only Tomato listings	Pass
B5	Filter by village	Only listings from that village	Pass
B6	Filter by price range	Only listings within the selected range	Pass
B7	Default card view	Product cards with price per unit	Pass
B8	Switch to table view	Tabular data with formatted prices	Pass
B9	Export listings to CSV	CSV file downloaded with current results	Pass

Table 3.3: Orders and Stock Test Cases

ID	Test Case	Expected Result	Status
C1	Buyer places a valid order	Order row created	Pass
C2	Stock decrement after an order	Listing quantity reduced by the ordered quantity	Pass
C3	Order reduces stock to zero	Listing status changed to sold_out	Pass
C4	Order exceeds available stock	Error: Insufficient stock	Pass
C5	Zero or negative quantity	Error: quantity must be greater than zero	Pass
C6	Order total calculation	Total equals quantity multiplied by price	Pass
C7	Valid transition pending to confirmed	Status updated and history row written	Pass
C8	Invalid transition pending to delivered	Database raises Invalid status transition	Pass
C9	Audit trail after a status change	Row in order_status_history with actor and timestamp	Pass
C10	Two buyers view their orders	Each buyer sees only their own orders	Pass
C11	Farmer views orders	Only orders on the farmer listings are shown	Pass

Table 3.4: Crop Advisor Test Cases

ID	Test Case	Expected Result	Status
D1	Valid soil and climate inputs	Crops matching at least four parameters, ranked	Pass
D2	Match percentage calculation	match_pct equals matched parameters divided by six	Pass
D3	Season filter set to Kharif	Only crops tagged for Kharif	Pass
D4	Inputs with no suitable crop	Empty result handled with a message	Pass
D5	Revenue and outlook fields	Revenue range and market outlook displayed	Pass
D6	Live weather lookup	Weather card populated for the entered place	Pass

Table 3.5: Reports and Tools Test Cases

ID	Test Case	Expected Result	Status
E1	Revenue per farmer	Sum of paid payments for the selected farmer	Pass
E2	Most active buyers	Buyers ranked by total volume	Pass
E3	Price trend for a crop	Current, minimum and maximum prices with chart	Pass
E4	Unsold stock for 15 or more days	Listings with harvest date older than fifteen days	Pass
E5	Average farmer rating	Average score and count per farmer	Pass
E6	Pending payments	Payments with status pending	Pass
E7	Profit calculator	Computed profit, margin and yield	Pass
E8	Scheme eligibility inputs	Indicative scheme matches displayed	Pass
E9	PDF export	PDF generated and downloaded	Pass

The negative tests were particularly useful. They confirmed that invalid Aadhaar formats never reach the database, that an order cannot exceed the available quantity, and that an illegal status jump is rejected by the function rather than silently accepted. The mobile checks also exposed layout issues at narrow widths, which were corrected before the final build.

SECTION 4

REFLECTION AND LEARNING

4.1 Challenges Faced

The first challenge was migrating the data layer from a direct PostgreSQL connection to Supabase's PostgREST interface. Complex joins and interval filters that were straightforward in SQL could not be sent as arbitrary queries through the REST client. The solution was to move each reusable report into a database view or a set-returning function and call it as an RPC, which also improved consistency.

The second challenge was keeping stock correct under concurrent activity. Performing the decrement in the application would have created a race condition between two buyers. Implementing it as an AFTER INSERT trigger ensured that every order, regardless of its origin, updated the listing atomically. The order lifecycle posed a similar problem: allowing the client to set any status would have made the audit trail meaningless, so the transition rules were centralised in update_order_status().

Authentication required careful handling. Storing an Aadhaar number, even in a course project, would have been inappropriate. The team implemented a one-way salted hash, restricted the login functions to return only an identifier and a name, and verified that the raw number never appeared in the interface or in logs. A fourth challenge was deployment: exposing a locally hosted application through a public tunnel required attention to certificates, process supervision and mobile layout.

Finally, the data set itself was a challenge. The reports look convincing only when the seed data contains enough variation in dates, quantities, ratings and payments. Preparing a realistic seed script made the views and charts meaningful and exposed edge cases such as a listing whose quantity reaches zero.

4.2 Key Learnings

The project provided practical experience in designing and operating a relational database as part of a complete application. The main technical and professional lessons were:

- Normalisation is not an academic exercise: separating listings, orders, payments, ratings and history removed duplication and made the queries simpler.

- Constraints and triggers belong in the database because they protect every client, including scripts and future applications.
- Views and stored functions are an effective way to centralise reporting logic and keep the API thin.
- A clear order lifecycle and an append-only audit trail make a transactional system easier to reason about and to defend.
- Security requirements such as hashing and least-privilege access must be designed at the start, not added after the interface is built.
- Testing negative cases is as important as testing the happy path; most defects appeared in invalid input and illegal transitions.
- Responsive design and deployment are part of the engineering work, not an afterthought.
- Version control, seed data and a repeatable deployment recipe make the project reproducible.

4.3 Suggestions for Improvement

Several improvements would make the system more complete and closer to a production marketplace:

- Integrate a payment gateway and reconcile payment status with the order lifecycle.
- Restore stock when an order is cancelled, using a second trigger or an explicit compensating transaction.
- Add notifications and a messaging channel so that farmers and buyers can coordinate delivery.
- Allow ratings only for delivered orders and add seller ratings for buyers.
- Replace seeded price history with a scheduled feed from a real market-price source.
- Introduce a scheduled job for backups, data retention and archival of old orders.
- Add an administrative dashboard for scheme verification, dispute handling and catalogue moderation.
- Improve the mobile experience with a progressive web app and offline-friendly caching.

- Expand the automated test suite to cover every route and function in continuous integration.

4.4 Programme Outcomes, PSOs and SDGs

The following tables map the project to the outcomes it directly addresses. The mapping is limited to the POs, PSOs and SDGs for which there is direct evidence in the implementation or documentation.

Table 4.1: Programme Outcome Mapping

PO	Outcome	How the project addresses it
PO1	Engineering Knowledge	Applies database and software engineering concepts through a normalised schema, triggers, views, stored functions and a working Next.js/FastAPI application.
PO2	Problem Analysis	Analyses farmer market-access and crop-selection problems, then converts them into role-based requirements and test cases.
PO3	Design/Development of Solutions	Designs an integrated marketplace with listings, orders, stock automation, reports, Crop Advisor and scheme explorer.
PO5	Engineering Tool Usage	Uses Next.js, React, TypeScript, FastAPI, Python, Supabase and Playwright for development, testing and documentation.
PO6	The Engineer and Society	Addresses agricultural marketing and farmer access through direct trading, price transparency and scheme information.
PO8	Ethics	Protects identity data by storing only a salted SHA-256 hash, applying role checks and never returning raw Aadhaar values.

Table 4.2: Programme Specific Outcome Mapping

PSO	Outcome	How the project addresses it
PSO1	Analyse and design database-backed systems	Implements a normalised relational schema with triggers, views, stored functions and reporting logic.
PSO2	Use modern tools to design and communicate	Applies Next.js, React, TypeScript, FastAPI, Python, Supabase and Playwright, and documents the design and tests.

Table 4.3: Sustainable Development Goal Mapping

SDG	Target	Project contribution
SDG 2	Zero Hunger (2.3, 2.c)	Direct farmer-to-buyer marketplace, price transparency, price history and crop advisory support better farmer income and market information.
SDG 8	Decent Work and Economic Growth (8.3)	Digital listings, orders, payment tracking, revenue reporting and scheme discovery support rural entrepreneurship.
SDG 9	Industry, Innovation and Infrastructure (9.3, 9.c)	Low-cost digital marketplace infrastructure built with Next.js, FastAPI and PostgreSQL improves rural access to ICT.
SDG 10	Reduced Inequalities (10.1, 10.2)	Focus on small and marginal farmers, local buyers and transparent market data reduces information asymmetry.
SDG 1	No Poverty (supporting)	Better market access and income tracking support farmer livelihoods.
SDG 12	Responsible Consumption and Production (supporting)	Inventory, low-stock and sales reporting support more efficient stock planning.
SDG 13	Climate Action (supporting)	Crop advisory uses soil, moisture and season inputs to support informed, climate-sensitive crop selection.

4.5 Conclusion

AgriConnect demonstrates how a carefully designed relational database can support a practical agricultural marketplace. Farmers can list produce, manage stock and process orders; buyers can search, compare and purchase directly; and both roles receive transparent reports and advisory tools. The database enforces the rules that matter: primary and foreign keys preserve relationships, CHECK constraints reject invalid data, a trigger keeps stock consistent, a stored function guards the order lifecycle, and views and functions produce the reports.

The project also shows the value of separating concerns. The Next.js interface, the FastAPI service and the PostgreSQL schema can evolve independently while sharing one source of truth. Aadhaar authentication, although simplified for a course project, follows the correct principle of storing a one-way hash rather than the raw identity number. The result is a coherent, testable and reproducible application that meets its objectives and provides a solid foundation for the improvements listed above.

SECTION 5

REFERENCES

[1] PostgreSQL Global Development Group, PostgreSQL 16 Documentation, 2024. Available: https://www.postgresql.org/docs/

[2] Supabase, Supabase Documentation: Database, Auth and PostgREST, 2025. Available: https://supabase.com/docs

[3] PostgREST, PostgREST Documentation, 2025. Available: https://postgrest.org/

[4] FastAPI, FastAPI Documentation, 2025. Available: https://fastapi.tiangolo.com/

[5] Vercel, Next.js Documentation, 2025. Available: https://nextjs.org/docs

[6] Open-Meteo, Open-Meteo Weather API Documentation, 2025. Available: https://open-meteo.com/en/docs

[7] OpenStreetMap Foundation, Nominatim Search API Documentation, 2025. Available: https://nominatim.org/release-docs/latest/api/Overview/

[8] National Informatics Centre, eNAM: National Agriculture Market, Government of India. Available: https://enam.gov.in/

[9] Department of Agriculture & Farmers Welfare, Government of India. Available: https://agricoops.nic.in/

[10] National Institute of Standards and Technology, FIPS PUB 180-4: Secure Hash Standard (SHS), 2015.

[11] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2017.

[12] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[13] Python Software Foundation, Python 3 Documentation, 2025. Available: https://docs.python.org/3/

[14] fpdf2, fpdf2 Documentation, 2025. Available: https://pyfpdf.github.io/fpdf2/